UD 2: Programación multihilo

Programación de Servicios y Procesos
DAM2

Contenidos y objetivos

Contenidos

- Hilos. Estados de un hilo
- Gestión de hilos
- Creación de hilos en Java
- Compartir información entre hilos
- Sincronización de hilos
- Gestión de prioridades

Objetivos

- Conocer las características de hilos en Java
- Crear y gestionar hilos
- Crear programas para compartir información entre hilos
- Crear programas formados por varios hilos sincronizados

Introducción

Los hilos los podríamos definir como diferentes líneas de ejecución de un mismo proceso, de modo que estamos refiriéndonos a un modo de programación concurrente, en la que los hilos comparten un espacio de memoria dentro del contexto de otro programa, mientras que los procesos generalmente mantienen su propio espacio de direcciónes y entorno de operaciones. A los hilos se les llama procesos ligeros.

Los hilos no pueden ejecutarse solos, necesitan la supervisión de un proceso padre para ejecutarse.

Utilidad: Realización de programas que tengan que llevar a cabo tareas simultáneamente.

Hilos en Java: la clase Thread

La clase Thread

- Creación de una clase que herede de la clase Thread
- Sobreescritura del método run()
- Llamada desde el programa padre mediante método start();

- Ejemplo 1: HiloEjemplo1
- Ejemplo 2: HiloEjemplo1_V2 y UsaHiloEjemplo_V2

Hilos en Java: la clase Thread

Métodos útiles

Métodos	Misión
start()	Hace que el hilo comience la ejecución. La máquina virtual de Java llama al método run() de este hilo
boolean isAlive()	Comprueba si el hilo está vivo
sleep(long mils)	Hace que el hilo actualmente en ejecución pase a dormir temporalmente durante el número de milisegundos especificado.
run()	Constituye el cuerpo del hilo. Es llamado por el método start() después de que el hilo apropiado del sistema se haya inicializado. Si el método run() devuelve el control, el hilo se detiene. Es el único método de la interfaz Runnable
String toString()	Devuelve una representación del hilo (con el nombre, la prioridad y el grupo de hilos)
long getId()	Devuelve el identificador del hilo
void yield()	Hace que el hilo actual de ejecución pare temporalmente y permita que otros hilos se ejecuten

Hilos en Java: la clase Thread

Métodos útiles

Métodos	Misión
String getName()	Devuelve el nombre del hilo
setName(String name)	Cambia el nombre de este hilo, asignándole el especificado como argumento
int getPriority()	Devuelve la prioridad del hilo
setPriority(int p)	Cambia la prioridad del hilo
void interrupt()	Interrumpe la ejecución del hilo
boolean interrupted()	Comprueba si el hilo ha sido interrumpido
Thread currentThread()	Devuelve una referencia al objeto hilo que se está ejecutando actualmente

Hilos en Java: la clase Thread

Métodos útiles

Métodos	Misión
boolean isDaemon()	Comprueba si el hilo es un hilo Daemon. Los hilos daemon o demonio son hilos de prioridad baja que normalmente se ejecutan en segundo plano. Un ejemplo de demonio es el garbage collector
setDaemon(boolean on)	Establece este hilo como hilo Daemon, asignando el valor true, o como hilo de usuario, pasando el valor false
void stop()	Detiene el hilo. Este método está en desuso.
int activeCount()	Este método devuelve el número de hilos activos en el grupo de hilos del hilo actual
Thread.State getState()	Develve el estado del hilo: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED

https://docs.oracle.com/javase/10/docs/api/java/lang/Thread.html

Hilos en Java: la clase Thread

- Ejemplo 3: HiloEjemplo2
- Ejemplo 4: HiloEjemplo2Grupos

- Actividad 2.1: Crea dos clases hilos Java que extiendan de la clase Thread. Uno de los hilos debe visualizar en pantalla en un bucle infinito la palabra TIC y el otro hilo la palabra TAC. Dentro del bucle utiliza el método sleep() para que dé tiempo a ir leyendo lo que van escribiendo estos hilos. Añadir bloques try-catch (para capturar InterrumpedException). Crea un programa main() que haga uso de los hilos anteriores.